UNIVERSITÉ DE SHERBROOKE

Faculté des sciences
2500, boul. de l'Université
Sherbrooke (Québec) J1K 2R1
CANADA

Département d'informatique
+1 (819) 821-7030
+1 (819) 821-8200
http://www.usherbrooke.ca/informatique/

IFT187 – Éléments de base de données

Examen #1 (sommatif)

- Administration prévue 2026-10-14

Notes et consignes

- Cet examen d'une durée de 120 minutes est individuel. Il comporte six questions.
- La correction est basée, entre autres, sur le fait que chacune des réponses soit claire (c'est-à-dire lisible et compréhensible), exacte (c'est-à-dire précise et sans erreurs), complète (c'est-à-dire comprenant tous les éléments requis et tous les éléments nécessaires) et concise (c'est-à-dire sans éléments superflus).
- Aucune documentation personnelle n'est autorisée.
- Aucun équipement informatique, électronique ou de communication n'est autorisé.
- **Justifier les réponses.**
- **Remettre le questionnaire et le cahier avant de quitter la salle d'examen.**

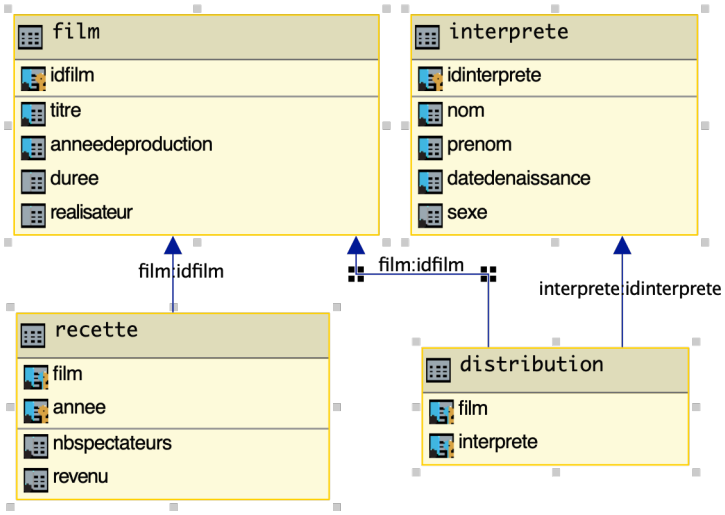
Pondération

Question	**Q1**	**Q2**	**Q3**	**Q4**	**Q5**	**Q6**	**Total**
Poids	/ 20	/ 30	/ 30	/ 10	/ 10	/ 10	/ 110

1 Programmation SQL (LDD)

Mise en contexte

Une société commerciale désire assurer la diffusion de films sur le territoire du Nunavik. Elle désire conserver un inventaire des principales caractéristiques des films dont elle possède les droits. Elle désire notamment conserver la liste des interprètes (comédiens) pour chaque film ainsi que le nombre des spectateurs et les revenus engendrés chaque année.



Question

Écrire le script de création SQL des tables requises, conformément au diagramme relationnel ci-contre. Spécifier clairement le prédicat de chaque table. Justifier le choix du type des attributs, lorsqu'approprié. Traiter adéquatement les clés candidates, les clés référentielles et les contraintes requises. Indiquer clairement vos hypothèses.

2 Évaluation de requêtes SQL (LMD)

Mise en contexte

Soit le schéma Provik et les données en annexe.

Questions

Pour chacune des requêtes suivantes :

(a) décrire en une phrase ou deux ce qui est calculé ;
(b) calculer et transcrire le contenu de la table résultante.

[2.1]

```
SELECT codeP, nom, budget
FROM Projet
WHERE cat = 'dev'
```

[2.2]

```
SELECT cat, COUNT(DISTINCT codeP)
FROM Projet
GROUP BY cat
```

[2.3]

```
SELECT Projet.codeP, COUNT(DISTINCT NMP)
FROM Projet LEFT JOIN Tache USING (codeP)
GROUP BY Projet.codeP
```

[2.4]

```
SELECT DISTINCT Projet.codeP, P.codeP
FROM Projet JOIN Projet AS P USING (fin)
WHERE (Projet.codeP < P.codeP)
ORDER BY Projet.codeP, P.codeP
```

[2.5]

```
WITH
A AS
  ( SELECT codeP, NMP
    FROM Membre
      JOIN Tache USING (NMP)
      JOIN Projet USING (codeP)
    WHERE embauche < '2015-01-01'
  ),
B AS
  (SELECT codeP FROM A GROUP BY codeP HAVING COUNT(*) > 1 )
SELECT codeP, NMP, nom, prenom, embauche
FROM A NATURAL JOIN B NATURAL JOIN Membre
ORDER BY codeP, embauche DESC
```

[2.6]

```
WITH
X AS
  ( SELECT DISTINCT NMP, codeP, MAX(part) AS total
    FROM Tache JOIN Membre USING (NMP)
    GROUP BY NMP, codeP
  )
SELECT DISTINCT X1.NMP
FROM X AS X1
WHERE NOT EXISTS
  ( SELECT *
    FROM X AS X2
    WHERE (X1.codeP = X2.codeP) AND (X1.NMP <> X2.NMP) AND (X1.total > X2.total)
  )
```

3 Programmation de requêtes SQL (LMD)

Mise en contexte

Soit le schéma et les données en annexe.

Questions

Proposer une requête SQL pour chacune des demandes suivantes :

[3.1] Quelle est la durée de la tâche la plus longue réalisée au mois de janvier 2017 (tous projets confondus) ? Donner la valeur.

[3.2] Quels sont les membres qui ont travaillé dans un projet en dehors des dates de début et de fin de celui-ci ? Donner le NMP, le nom, le prénom de ceux-ci.

[3.3] Quels sont les projets ayant utilisé plus de trois membres différents ? Donner le code, le nom, la catégorie, le nombre de membres employés de chacun de ces projets.

[3.4] Quels sont les projets ayant consommé un nombre d'heures plus grand que la moyenne des heures consommées par projet ? Donner le code, le nom, la catégorie et le nombre d'heures consommées de chacun de ces projets.

4 Déclaration de fonctions

Mise en contexte

On désire connaitre le nombre de films disponibles pour un réalisateur donné.

Question

Écrire une fonction SQL qui retourne ce résultat.

5 Modèle relationnel

Mise en contexte

Le modèle relationnel définit la valeur d'une relation comme un ensemble de tuples.

Questions

[5.1] En quoi cela diffère-t-il du modèle SQL ?

[5.2] Donner au moins deux conséquences pratiques imputables à cette différence.

6 Intégrité référentielle

Mise en contexte

Les clés référentielles (étrangères) sont un mécanisme important de modélisation relationnelle. Elles reposent sur la définition préalable de clés candidates. La clé référentielle établit un lien entre les attributs d'une relation et la clé candidate d'une autre relation.

Question

[6.1] Que se passe-t-il lors de la suppression d'un tuple dont la valeur de la clé candidate est référée par la clé référentielle d'un tuple présent dans la base de données ?

[6.2] Quels sont les mécanismes offerts par SQL pour faire face à cette situation ?

Annexe

La société Provik assure la gestion de projets sur le territoire du Nunavik. Elle désire faire le suivi des tâches réalisées par les membres du personnel au sein de chacun des projets.

Un projet est identifié par un code (unique) de projet et caractérisé par un nom, une date de début, une date de fin, une catégorie et un budget (nombre d'heures de travail allouées). Un membre du personnel est identifié par un numéro (unique) et caractérisé par son nom, son prénom et sa date d'embauche. Chaque membre du personnel consigne à la fin de chaque journée ouvrée sa participation à chaque projet auquel il a participé en donnant le nombre d'heures consacrées et la description des tâches qu'il a accomplies au sein du projet. Considérer les trois tables décrites par les énoncés suivants :

```
CREATE DOMAIN Numero_MP
  TEXT
  CHECK (VALUE SIMILAR TO 'E[0-9]{3}');
CREATE DOMAIN Nom_personne
  TEXT
  CHECK (VALUE SIMILAR TO '[[:alpha:][:space:]''-]+');
CREATE DOMAIN Code_projet
  TEXT
  CHECK (VALUE SIMILAR TO 'P[0-9]{3}');
CREATE DOMAIN Categorie
  TEXT
  CHECK (VALUE SIMILAR TO '[A-Za-z0-9]{3}');

CREATE TABLE Membre (
    NMP         Numero_MP      NOT NULL,     -- numéro de membre du personnel
    nom         Nom_personne   NOT NULL,
    prenom      Nom_personne   NOT NULL,
    embauche    DATE           NOT NULL,     -- date d'embauche
    CONSTRAINT Membre_cc0     PRIMARY KEY (NMP)
);
CREATE TABLE Projet (
    codeP   Code_projet      NOT NULL,    -- code de projet
    nom     VARCHAR(32)      NOT NULL,
    budget  NUMERIC(6)       NOT NULL,    -- budget total en heures
    cat     Categorie        NOT NULL,
    debut   DATE             NOT NULL,
    fin     DATE             NOT NULL,
    CONSTRAINT Projet_cc0    PRIMARY KEY (codeP),
    CONSTRAINT Projet_budget CHECK (1.0 <= budget),
    CONSTRAINT Projet_dates  CHECK (debut <= fin)
);
CREATE TABLE Tache (
    NMP       Numero_MP    NOT NULL,
    codeP     Code_projet  NOT NULL,
    jour      DATE         NOT NULL,
    part      NUMERIC(2)   NOT NULL,    -- participation en heures
    description TEXT       NOT NULL,
    CONSTRAINT Tache_cc0   PRIMARY KEY (NMP, codeP, jour),
    CONSTRAINT Tache_ce1   FOREIGN KEY (NMP) REFERENCES Membre,
    CONSTRAINT Tache_ce2   FOREIGN KEY (codeP) REFERENCES Projet,
    -- un membre du personnel ne peut travailler plus de 16 heures par jour
    CONSTRAINT Tache_part  CHECK (part BETWEEN 1 AND 16)
);
```

Initialement, les données sont les suivantes :

Membre

nmp	nom	prenom	embauche
E001	Smith	Paul	2015-03-01
E002	Dupont	Jean	2015-01-01
E003	Durand	Maude	2014-01-01
E004	Clark	Joe	2013-08-01
E005	El Khoury	Alexandre	2015-06-01

Projet

codep	nom	budget	cat	debut	fin
P001	Portail	100	dev	2014-01-01	2016-06-30
P002	Boutique	50	exp	2015-01-01	2016-06-30
P003	Velox	20	dev	2016-01-01	2016-06-30
P004	Velox	160	maj	2017-01-01	2017-06-30
P005	Plox	1000	dev	2017-01-01	2018-08-30
P006	Redox	10	maj	2017-06-01	2017-06-30

Tâche

nmp	codep	jour	part	description
E001	P001	2014-01-01	3	anomalie #03
E001	P002	2015-01-01	12	test #05
E001	P003	2016-01-01	4	conception IRT
E001	P004	2017-01-01	12	anomalie #12
E001	P005	2017-01-01	11	documentation
E001	P005	2017-01-02	11	suivi
E002	P001	2014-01-02	8	anomalie #33
E002	P002	2015-01-02	4	test #33
E003	P002	2016-01-02	12	suivi
E004	P002	2016-01-02	7	test #12
E004	P004	2017-01-02	3	documentation
E004	P005	2017-01-02	4	suivi

Notes de cours

1. Types, domaines, tables et contraintes

Types usuels

Booléen	BOOLEAN
Texte	CHAR(n) • VARCHAR(n) • TEXT
Entier	SMALLINT • INTEGER • BIGINT
Décimal exact	NUMERIC(p,s) — p : chiffres totaux; s : chiffres après la virgule
Temporel	DATE • TIME • TIMESTAMP • INTERVAL

Opérateurs et fonctions utiles

Motif	expression SIMILAR TO 'motif'
Intervalle	expression BETWEEN borne1 AND borne2
Appartenance	expression IN (valeur1, valeur2, ...)
NULL	expression IS NULL / IS NOT NULL
Sous-chaîne	substr(texte, début, longueur)
Temps courant	current_date • current_timestamp
Différence temporelle	age(ts0, ts1) • age(ts)
Composante temporelle	extract(YEAR | MONTH | DAY FROM valeur)

Définir un domaine

```
CREATE DOMAIN nom_domaine [AS] type
  [DEFAULT expression]
  [[CONSTRAINT nom_contrainte] {NOT NULL | NULL | CHECK (expression)} ...];
```

Créer une table

```
CREATE TABLE nom_table (
  colonne1 type [NOT NULL] [DEFAULT expression],
  colonne2 type [NOT NULL],
  ...
  CONSTRAINT nom_pk PRIMARY KEY (colonne1, ...),
  CONSTRAINT nom_uq UNIQUE (colonne, ...),
  CONSTRAINT nom_fk FOREIGN KEY (colonne, ...)
      REFERENCES table_referee (colonne, ...),
  CONSTRAINT nom_ck CHECK (condition)
);
```

2. Requêtes SQL

Structure générale

```
[WITH nom AS (requête), ...]
SELECT [DISTINCT] expression [AS alias], ...
FROM table [AS alias] [JOIN ...]
[WHERE condition]
[GROUP BY expression, ...]
[HAVING condition]
[ORDER BY expression [ASC | DESC], ...]
[LIMIT n] [OFFSET n];
```

Groupement

```
Groupement ::=
 [ GROUP BY { expression ... , } ]
 [ HAVING { condition ... , } ]
```

Restriction

```
restriction ::=
 [ WHERE condition* ]

condition ::=
    quantification_gen
  | quantification_com
  | expression
```

Quantification générale

```
quantification_gen ::=
 [NOT] EXISTS (requête)
```

Quantification de comparaison

```
quantification_com ::=
 expression opComp [ANY | ALL] ensemble

opComp ::=
    = | <> | < | <= | > | >=

ensemble ::=
    ( requête ) | ( listeDeValeurs )
```

Ordonnancement

```
Ordonnancement ::=
 [ ORDER BY { expression [ ordre ] [ t_null ] ... , } ]
 [ LIMIT { nombre | ALL } ] [ OFFSET nombre ]

order ::= ASC | DESC | USING opérateur

t_null ::= NULLS { FIRST | LAST }
```

Jointures

```
FROM A JOIN B ON A.x = B.y
FROM A JOIN B USING (x, y)
FROM A NATURAL JOIN B
FROM A LEFT JOIN B  ON ...
FROM A RIGHT JOIN B ON ...
FROM A FULL JOIN B  ON ...
```

Agrégations

Fonction	Rôle
COUNT(*)	Nombre de lignes
COUNT(col)	Valeurs non nulles
COUNT(DISTINCT col)	Valeurs distinctes non nulles
SUM(col)	Somme
AVG(col)	Moyenne
MIN(col) / MAX(col)	Minimum / maximum

Opérations ensemblistes

```
requête1 UNION requête2   •   requête1 INTERSECT requête2   •   requête1 EXCEPT requête2
```

Mise à jour

```
UPDATE nom_table [AS alias]
SET colonne = expression, ...
[FROM table2, ...]
[WHERE condition];
```

3. Fonctions, procédures et vues

Définir une fonction SQL

```
CREATE [OR REPLACE] FUNCTION nom ([parametre type, ...])
RETURNS type_retour
LANGUAGE SQL AS
$$
  ...;
$$;
```

Définir une procédure SQL

```
CREATE [OR REPLACE] PROCEDURE nom ([parametre type, ...])
LANGUAGE SQL AS
$$
  ...;
$$;
```

Définir une vue

```
CREATE [OR REPLACE] VIEW nom_vue [(colonne1, colonne2, ...)]
AS
  requête;
```